

eQMS - Permission model

1. Overview

The system combines **RBAC (Role-Based Access Control)** and **ABAC (Attribute-Based Access Control)** to enforce strict **GxP-compliant access** to documents, historical versions, tenants, and users.

- **RBAC:** Assigns users to roles; roles carry a set of base permissions.
- **ABAC:** Adds conditional rules based on **attributes** of the user, document, or environment.
- **Field-level ABAC:** Optional restrictions on individual fields within a document.

2. Permissions

Permissions define **actions users can perform**. They are **granular**, can be combined into roles, and are tenant-scoped.

2.1 Core Permissions

Permission	Description	Typical Usage
<code>document.read</code>	View document metadata	Assigned to Viewer, Editor, Auditor
<code>document.write</code>	Create or update documents	Assigned to Editor, Approver
<code>document.delete</code>	Delete or archive documents	Assigned to Admin
<code>document.approve</code>	Approve document for release	Assigned to Approver
<code>document.version.view</code>	View historical versions	Assigned to Viewer, Editor, Approver
<code>document.content.download</code>	Download file content	Viewer, Editor, Approver
<code>user.read</code>	Read user info within tenant	Admin, Auditor
<code>user.write</code>	Create/update tenant users	Admin
<code>role.read</code>	Read role definitions	Admin
<code>role.write</code>	Create/update/delete roles	Admin
<code>tenant.read</code>	Read tenant details	Admin, Auditor

<code>tenant.write</code>	Create/update tenant	Admin
<code>audit.read</code>	Read audit trail	Auditor
<code>audit.export</code>	Export audit trail	Auditor

3. Roles

Roles are **collections of permissions**. Roles can be **global system roles** (for admins) or **tenant-specific**.

Role	Permissions	Notes
Viewer	<code>document.read</code> , <code>document.version.view</code> , <code>document.content.download</code>	Can only see documents and historical versions.
Editor	All Viewer permissions + <code>document.write</code>	Can create/update documents but not approve or delete.
Approver	Editor permissions + <code>document.approve</code>	Can approve documents for release.
Admin	All document, user, role, tenant permissions	Full control over tenant resources.
Auditor	<code>audit.read</code> , <code>audit.export</code> , <code>document.read</code> , <code>document.version.view</code>	Can read documents and audit trail but cannot modify.

4. RBAC Workflow

1. User Assignment

- Users are assigned one or more roles within a tenant.
- Roles map to permissions.

2. Permission Check

- When a user calls an API, the system checks whether **the required permission exists in any of the user's roles**.

3. Example: Get Document

```
1 User has roles: ["Viewer"]
2 Required permission: document.read
3 Check: Viewer → document.read → allowed
4
```

4. Example: Update Document

```
1 User has roles: ["Viewer"]
2 Required permission: document.write
3 Check: Viewer → no permission → denied (403)
4
```

5. ABAC (Attribute-Based Access Control)

RBAC alone is not enough for **GxP multi-tenant SaaS**, because we need **contextual rules**:

- **Document Attributes:** classification, ownerId, status, version
- **User Attributes:** roles, department, employment status, tenancy
- **Environment Attributes:** timeOfAccess, sourceIp

5.1 Example ABAC Rules

Rule	Description	Example
Classification-based access	Users can only read documents allowed for their clearance	User clearance = “Controlled” → can read doc with classification ≤ Controlled
Owner-based edit	Users can edit documents they own	User.id = Document.ownerId → can edit
Version restrictions	Certain historical versions only visible to Auditor	Document.version.status = “Released” → Viewer can see, else only Auditor
Field masking	Sensitive fields hidden based on role	Field <code>email</code> → shown only to Admin, hidden for Viewer

5.2 Field-level ABAC

- Only include fields allowed by ABAC in `fieldsReturned` for audit.
- Example:

```
1 {
2   "fieldsReturned": ["documentId", "title", "version"]
3 }
4
```

- If ABAC hides `classification`, it is omitted.

6. ABAC Evaluation Flow (for developers)

1. Single Document

- Retrieve metadata/content.
- Evaluate ABAC rules against document attributes + user attributes.
- Return only allowed fields and generate audit event with `fieldsReturned`.

2. List / Search

- Apply **ABAC filters at the database query** whenever possible to reduce data exposure.
- For remaining rules, filter in memory or in API layer.
- Generate **single audit event** capturing `fieldsReturned` and `itemsCount`.

7. Combined RBAC + ABAC Example

Scenario: Viewer trying to download a controlled document owned by another department:

1. RBAC Check

- Viewer role → has `document.content.download` permission 

2. ABAC Check

- Document classification = “Highly Restricted”
- User department ≠ document department → Denied 

3. Result

- 403 Forbidden returned
- Audit event logged for attempted access

8. Multi-Tenant Considerations

- RBAC/ABAC rules are **scoped per tenant**.
- Users cannot access documents from other tenants.
- Audit events include `tenantId` to ensure traceability.